

arc42 Architecture Documentation

Web Application Firewall (WAF)

с использованием методов ML

1. Введение и цели

Название системы: Web Application Firewall (WAF) с использованием методов ML

1.1 Цель системы

Система обеспечивает защиту веб-приложений от распространённых атак: SQL Injection, XSS, SSRF, XXE, Command Injection, Path Traversal, Business Logic Attacks и других угроз согласно OWASP Top 10. ML используется **в дополнение** к классическим правилам — для повышения точности детекции, снижения false positives и анализа поведенческих аномалий.

1.2 Цели по этапам

Курсовая работа — реализовано:

- WAFCore: приём, маршрутизация и проксирование HTTP-запросов
- SecurityEngine с детекторами: SQLi, XSS, Command Injection, SSRF, XXE, Path Traversal, File Inclusion, Cookie Tampering
- RuleManager с поддержкой CEL-выражений (cel-go) и горячей перезагрузкой правил
- Rate Limiting на базе Redis (Sliding Window)
- PluginManager: plugin-based цепочка детекторов
- EventPublisher: асинхронная публикация событий безопасности
- Structured logging (logrus/zap), метрики Prometheus

Дипломная работа — запланировано и обосновано:

- MLInferenceService (Python/gRPC): LightGBM + Isolation Forest + LSTM-Autoencoder — детали в разделе 5.7
- Feature extraction pipeline: ~150 признаков из запроса, сессии, поведения — детали в разделе 5.7
- Behavioral Analytics: per-endpoint и per-tenant baseline на основе LSTM-Autoencoder
- Event Streaming Platform: Kafka/NATS как backbone для decoupling компонентов
- Distributed State: Redis Cluster для Feature Store и Rate Limiting
- Kubernetes: горизонтальное масштабирование, CI/CD, ConfigStore (etcd)
- Multi-tenancy, SIEM/SOC-интеграция, Compliance (PCI DSS, GDPR)

2. Ограничения

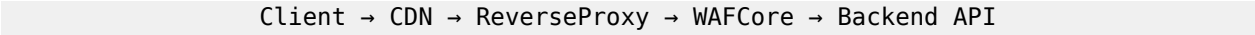
2.1 Технический стек

Компонент	Технология	Этап
WAF Core	Go 1.22+	Курсовая
Rule Language	CEL (cel-go) + YAML DSL	Курсовая
ML Inference	Python 3.11, gRPC, ONNX Runtime	Диплом
ML Models	LightGBM, Isolation Forest, LSTM-Autoencoder	Диплом
Event Streaming	Kafka / NATS JetStream	Диплом
State / Cache	Redis Cluster	Курсовая (базово)
Feature Store	Redis + custom schema	Диплом
Оркестрация	Kubernetes + ArgoCD	Диплом
Config Store	etcd / Consul	Диплом
Observability	Prometheus, Jaeger, Elasticsearch	Диплом

3. Контекст системы

3.1 Внешний контекст (целевая архитектура)

Система располагается между клиентом и backend API, проксируя весь трафик через несколько слоёв защиты.



3.2 Внешние участники

Участник	Тип	Взаимодействие
Client (браузер / API-клиент)	Внешний	HTTP/HTTPS запросы
CDN (Cloudflare / Akamai)	Внешняя инфраструктура	DDoS-защита, GeoBlocking, IP Blocking
ReverseProxy (Nginx / Envoy)	Инфраструктура	TLS Termination, Load Balancing, Header Normalization
Backend API	Внутренний сервис	Получает только разрешённые запросы
Identity Provider (OAuth2/JWT)	Внешний сервис	Валидация JWT-токенов (диплом — полный OAuth2)
Threat Intelligence (TAXII/STIX, MISP)	Внешний	Обновление IOC, репутация IP
SIEM / SOC	Внешний	Получение security events
Management UI / CLI	Оператор	Управление правилами, конфигурацией

3.3 Контекст курсовой работы

На текущем этапе реализован упрощённый поток без CDN и внешних интеграций. JWT-валидация реализована базово (верификация подписи), полноценный IdentityProvider — в дипломе.



4. Стратегия решения

Принцип	Описание	Этап
Модульность	Plugin-based детекторы через PluginManager; каждый детектор — независимый модуль	Курсовая
Rule Language	Язык правил на базе CEL (cel-go): читаемо, типизировано, без compile-time зависимостей	Курсовая
Hybrid Decision Engine	classic_score → ML только в "серой зоне" (0.3–0.85): экономия latency	Диплом
Разделение ответственности	WAFCore, SecurityEngine, MLInferenceService — независимые компоненты с чёткими gRPC-контрактами	Оба этапа
Event-driven	EventPublisher → Kafka/NATS → Consumers: decoupling аналитики от ядра	Диплом
Масштабируемость	Stateless WAFCore, горизонтальное масштабирование через Kubernetes HPA	Диплом
Security by design	mTLS, JWT/OAuth2, RBAC, Schema validation, tamper-proof audit log	Оба этапа

5. Структура (Building Block View)

5.1 Уровень 1 — Полная целевая архитектура

Пакет	Технология	Назначение	Этап
Traffic Ingress Layer	CDN + Nginx/Envoy	Edge-фильтрация, TLS, load balancing	Диплом
Core WAF System	Go	Ядро: обработка запросов, оркестрация	Курсовая
Security Engine	Go	Анализ угроз, RuleManager (CEL)	Курсовая
Vulnerability Detection	Go	8 детекторов атак (SQLi, XSS и др.)	Курсовая
ML Platform	Python + gRPC	LightGBM + IsolationForest + LSTM-AE	Диплом
Behavioral Analytics	Go + ML	Per-endpoint/per-tenant baseline, аномалии	Диплом
State Management	Redis Cluster	Rate limiting, Feature Store, сессии	Частично
Event Streaming	Kafka / NATS	Message Bus, event sourcing, replay	Диплом
Identity & Access Mgmt	Go	JWT верификация → полный OAuth2+RBAC	Частично
Forensics & Compliance	Go	Tamper-proof логи, PCI DSS / GDPR отчёты	Диплом
Observability Stack	Prometheus/Jaeger/ES	Метрики, distributed tracing, логи	Диплом
Multi-Tenancy	Go	TenantIsolator, ResourceQuotas, BillingEngine	Диплом
Management Plane	Go + REST/CLI	Управление правилами CEL, конфигурацией	Курсовая (базово)
Deployment & Infra	Kubernetes + CI/CD	HPA, ArgoCD, etcd, Vault	Диплом

5.2 Уровень 2 — Core WAF System

WAFCore

Поле / Метод	Тип	Описание
config	Config	Конфигурация (загрузка из etcd, hot reload)
requestProcessor	RequestProcessor	Обработка входящих запросов
responseProcessor	ResponseProcessor	Обработка ответов backend
pluginManager	PluginManager	Цепочка детекторов + gRPC ML-клиент
securityEngine	SecurityEngine	RuleManager (CEL) + детекторы
metrics	MetricsExporter	Prometheus: latency, throughput, block_rate
logManager	LogManager	Structured logging (zap/logrus)
eventPublisher	EventPublisher	Async публикация в Kafka/NATS
HandleRequest(req)	method → Response	Основной pipeline: context → engine → decision
ReloadConfig()	method	Hot reload из etcd без перезапуска

RequestContext

Поле	Тип	Описание
RequestID	UUID	Уникальный ID для distributed tracing
TenantID	string	Идентификатор тенанта
IP	string	IP клиента + репутация (TI feed)
JWTClaims	map	Декодированные claims (sub, roles, scope)
RiskScore	float	Итоговый score: 0.0 (safe) — 1.0 (attack)
MLScore	float	Score от MLInferenceService (если запрошен)
Fingerprint	string	TLS fingerprint + behavioral fingerprint
SessionID	string	ID сессии для behavioral tracking
IsBlocked()	bool	Признак блокировки
GetThreatLevel()	Level	LOW / MEDIUM / HIGH / CRITICAL

5.3 Security Engine — Rule Manager

Язык правил реализован на базе **cel-go** (Google Common Expression Language). Выбор обоснован в разделе 9.

RuleManager

- Хранит правила в YAML-формате: name, description, severity, CEL-expression, action
- RuleCompiler: компилирует CEL-выражения в AST при загрузке — JIT не нужен (CEL уже компилируется)
- Поддерживает горячее включение/отключение правил без перезапуска
- TestRule(rule, testData) — sandbox-тестирование; используется в CI/CD pipeline
- Adversarial Testing: набор mutation-тестов для каждого детектора (фаззинг через go-fuzz)

Пример CEL-правила для SQLi:

```
request.query.matches('( ?i)(union|select|insert|drop|exec|xp_') ) && request.entropy > 3.5
```

Тестирование детекторов

Тип тестирования	Инструмент	Что проверяется
Unit tests	go test + testify	Каждый детектор на известных payload'ax
Mutation testing	go-mutesting	Полнота покрытия: убийство мутантов
Fuzzing	go-fuzz / AFL++	Нахождение необработанных edge-cases
Adversarial testing	Собственный harness	Обфускация, двойное кодирование, bypass-техники
Performance tests	k6 / hey	Latency p99 < 10ms под нагрузкой 100K rps
Integration tests	Docker Compose + testify	End-to-end: запрос → решение → лог

5.4 Vulnerability Detection — детекторы

Все детекторы реализуют интерфейс **Rule** (Evaluate, GetSeverity, GetCategory). Цепочка выполняется через PluginManager.ExecuteChain(ctx).

Детектор	Атака	Техника детекции	Этап
SQLInjectionDetector	SQL Injection	CEL + regex + entropy analysis	Курсовая
XSSDetector	Cross-Site Scripting	CEL + DOM-pattern matching	Курсовая
CommandInjectionDetector	OS Command Injection	Shell metachar patterns + CEL	Курсовая
FileInclusionDetector	LFI / RFI	Path traversal patterns + protocol check	Курсовая
SSRFDetector	Server-Side Request Forgery	RFC1918 + metadata endpoint patterns	Курсовая
XXEDetector	XML External Entity	XML parser + DOCTYPE/ENTITY analysis	Курсовая
PathTraversalDetector	Directory Traversal	../ normalization + encoded variants	Курсовая
CookieTamperingDetector	Cookie Manipulation	HMAC verification + anomaly score	Курсовая
BusinessLogicDetector	BOLA, BFLA, Mass Assignment	ML-based (LightGBM) — требует обучения	Диплом

Business Logic Attacks (BOLA/BFLA) не покрываются регулярными выражениями по природе. Детекция реализуется через ML-модель (LightGBM), обученную на паттернах нарушения авторизации. Отдельного языка и компилятора не требуется.

5.5 State Management

Компонент	Backend	Назначение
RateLimitCache	Redis Cluster	Sliding Window rate limiting, latency <1ms
SessionStore	Redis Cluster	Хранение сессий, CSRF, session fingerprinting
AttackSignatureCache	Redis + Bloom Filter	Кэш скомпилированных сигнатур CEL, TTL management
FeatureStore	Redis Cluster	Per-endpoint baseline: mean/std latency, param counts, status distribution (диплом)
BehavioralBaseline	Redis + time-series	LSTM-Autoencoder baseline per-tenant/per-user (диплом)

5.6 Event Streaming — топики (диплом)

Категория	Топики
Security Events	waf.security.attack.detected, waf.security.attack.blocked, waf.security.anomaly.detected, waf.security.rate_limit.exceeded
ML Events	waf.ml.prediction, waf.ml.model.updated, waf.ml.training.completed, waf.ml.drift.detected
Audit Events	waf.audit.config.changed, waf.audit.rule.updated, waf.audit.admin.action
Performance	waf.metrics.latency, waf.metrics.throughput, waf.metrics.error.rate, waf.health.status
State Changes	waf.state.rule.changed, waf.state.config.updated, waf.state.tenant.modified

5.7 ML Platform — детальная архитектура (диплом)

Этот раздел отвечает на вопрос: "Какие именно модели? Какие признаки? Как обучается?"

5.7.1 Цели ML-компонента

- Повышение точности детекции — особенно Business Logic Attacks и zero-day
- Снижение false positives: уточнение решения в "серой зоне" (classic_score 0.3–0.85)
- Behavioral Analysis: обнаружение отклонений от нормального профиля (per-endpoint, per-tenant)
- Risk Score refinement: финальный score = 0.6 × classic + 0.4 × ML

5.7.2 Feature Engineering (~150 признаков)

Группа признаков	Признаки	Кол-во
Request Features	URL length, path depth, query length, param count; entropy URL/query/body/headers; special char ratios (%2e, <, >, ', ";"); SQL keyword score; XSS pattern score; HTTP method, Content-Type	~60
User-Agent & IP	Browser family, OS, device type, bot_score; IP reputation, ASN, country risk, req/min velocity; TLS fingerprint (JA3)	~25
Session & Behavioral	Request rate, time since last req; session duration, req/session; deviation от user's usual endpoints; time-of-day pattern	~30
Structural	JSON depth, XML complexity, GraphQL query depth; ratio encoded chars (single/double encoding, unicode normalization)	~20
Per-Endpoint Baseline	Отклонение от mean/std: latency, body size, param count, status codes — берётся из FeatureStore	~15

5.7.3 Модели

Задача	Модель	Обучение	Применение	Latency
Быстрый pre-filter	Isolation Forest	Unsupervised	Все запросы (серая зона)	<2ms
Классификация атак	LightGBM	Supervised	Серая зона: RF 0.3–0.85	<5ms
Behavioral anomaly	LSTM-Autoencoder	Unsupervised	Per-endpoint baseline deviation	<10ms
Bot detection	Random Forest + поведение	Supervised	User-Agent + session patterns	<3ms
Business Logic	LightGBM (BOLA/BFLA features)	Supervised	API endpoints с авторизацией	<5ms
Финальный Risk Score	Logistic Regression (meta)	Supervised	Стекинг всех сигналов	<1ms

5.7.4 Датасеты и обучение

- Начальный датасет: CSIC 2010 HTTP Dataset + CICIDS 2017/2018 + собственные атаки
- Нормальный трафик: синтетический генератор + реальный трафик с разметкой
- Active Learning: запросы из серой зоны (0.4–0.6) отправляются на разметку
- Offline training: Airflow/cron — переобучение при дрейфе (KL-divergence мониторинг)
- ONNX Runtime: модели экспортируются в ONNX для ускорения инференса в Python

5.7.5 Hybrid Decision Engine

```
if classic_score > 0.85 → BLOCK elif classic_score < 0.30 → ALLOW else → ML_score =  
MLInference(features) final_score = 0.6 × classic + 0.4 × ML if final_score > 0.75 →  
BLOCK
```

*ML вызывается только для 5-15% запросов ("серая зона"), что обеспечивает p99 latency
WAFCore < 10ms при ML latency < 25ms.*

5.7.6 Метрики качества ML

Метрика	Целевое значение	Инструмент мониторинга
Precision (по классам)	> 0.95 для SQLi/XSS, > 0.85 для BLA	MLflow + Prometheus
Recall	> 0.90 для всех классов атак	MLflow
F1-score	> 0.92 (macro avg)	MLflow
False Positive Rate	< 1%	Prometheus + AlertManager
ML Latency p99	< 25ms	Jaeger tracing
ML Coverage	5-15% всех запросов	Prometheus counter
Feature Drift	KL-divergence < 0.1	Evidently AI / custom

6. Data View — хранение данных

Раздел описывает, **где и как** хранятся все типы данных системы.

Тип данных	Хранилище	Формат	TTL / Retention
Правила безопасности	etcd (ConfigStore)	YAML + CEL AST	Без TTL, версионизируется
ML-модели	S3-совместимое хранилище	ONNX format	Версионирование (MLflow)
Rate Limit state	Redis Cluster	Counter + TTL	1 минута (sliding window)
Session data	Redis Cluster	JSON + HMAC	30 минут (idle)
Attack signatures	Redis + Bloom Filter	Binary / CEL AST	24 часа, refresh по событию
Feature Store	Redis Cluster	Float64 vectors	7–30 дней (baseline)
Behavioral baseline	Redis + time-series (InfluxDB)	Float64 + metadata	30–90 дней
Security events	Kafka → Elasticsearch	JSON structured	90 дней (hot), 1 год (cold)
Audit log	Elasticsearch + S3	JSON, tamper-proof	5 лет (PCI DSS требование)
ML training data	S3 + Data Catalog	Parquet	1 год, потом архив
Трейсы (tracing)	Jaeger → Elasticsearch	OpenTelemetry	7 дней
Метрики	Prometheus → Thanos	Time-series	13 месяцев

7. Runtime View

7.1 Основной pipeline обработки запроса

№	Компонент	Действие
1	ReverseProxy → WAFCore	HTTP-запрос; TLS уже terminated
2	WAFCore	Создаётся RequestContext (ID, IP, TenantID, Fingerprint)
3	WAFCore → JWT Validator	Верификация подписи JWT, заполнение JWTClaims
4	WAFCore → SecurityEngine	Передача контекста в движок
5	SecurityEngine → RuleManager	Компиляция и применение CEL-правил
6	RuleManager → PluginChain	SQLi → XSS → CMDi → SSRF → XXE → PathTraversal → Cookie → ...
7	SecurityEngine	Вычисление classic_score → RiskScore
8	Hybrid Decision Engine	Если $0.30 < \text{classic} < 0.85$ → вызов MLInferenceService (gRPC)
9	MLInferenceService	IsolationForest pre-filter → LightGBM → final_score
10	WAFCore	Решение: allow → проксировать / block → 403 + event
11	EventPublisher → Kafka	Async публикация security event (не блокирует ответ)
12	LogManager	Structured log с RequestID для tracing

7.2 Сценарий Rate Limiting

№	Компонент	Действие
1	SecurityEngine → RateLimiter	CheckQuota(TenantID, IP, endpoint, limit)
2	RateLimiter → Redis	INCR + EXPIRE — атомарная операция, latency <1ms
3a	Redis	Квота не превышена → Increment, продолжить pipeline
3b	Redis	Квота превышена → RateLimitExceeded, block + event
4	EventPublisher	waf.security.rate_limit.exceeded → Kafka

8. Deployment View

8.1 Курсовая работа

Аспект	Описание
Запуск	go run / go build; опционально docker-compose (WAFCore + Redis + mock backend)
Конфигурация	YAML-файл (config.yaml); поддержка env-переменных
Redis	Один инстанс; используется для Rate Limiting и Session Store
Тестирование	go test ./... ; k6 для load tests; curl/Postman для ручных проверок
Сборка/CI	GitHub Actions: lint → test → build → docker push

8.2 Дипломная работа

Компонент	Технология	Описание
Оркестрация	Kubernetes	WAFCore, MLService, Kafka, Redis в подах; namespace per tenant
CI/CD	ArgoCD + GitHub Actions	GitOps: BlueGreen/Canary deployment, automated rollback
ConfigStore	etcd	Хранение CEL-правил, конфигурации; Watch для hot reload
SecretManager	Vault (HashiCorp)	Ротация секретов, PKI для mTLS, шифрование at rest
Service Mesh	Istio	mTLS между сервисами, circuit breaker, traffic policies
Scaling	Kubernetes HPA	По метрикам: CPU, RPS, p99 latency
ML Training	Airflow + MLflow	Offline training pipeline; версионирование моделей
Edge	LightweightAgent	Упрощённый WAFCore без ML; синхронизируется с центром

9. Кросс-секционные концепции

9.1 Observability

Аспект	Инструмент	Компонент
Метрики	Prometheus + Alertmanager	MetricsExporter → Grafana дашборды (latency, block_rate, ML_coverage)
Логирование	Zap → Fluentd → Elasticsearch	Structured JSON с RequestID, TenantID, RiskScore
Трейсинг	Jaeger (OpenTelemetry)	Spans: WAFCore → SecurityEngine → ML → Backend
Алерты	AlertManager → PagerDuty	FPR > 1%, latency p99 > 10ms, ML drift > 0.1

9.2 Безопасность

- JWT верификация (курсовая) → полный OAuth2 + RBAC через AuthorizationEngine (диплом)
- mTLS между всеми внутренними сервисами (Istio + Vault PKI)
- Tamper-proof audit log: каждая запись подписывается HMAC, хранится в Elasticsearch + S3
- ConfigurationIntegrity: CEL-правила хранятся с цифровой подписью в etcd

9.3 Конфигурация и язык правил

- Язык правил: CEL (cel-go) — безопасный, типизированный, без side-effects
- Альтернатива Rego (OPA) рассматривалась — отклонена (см. раздел 10)
- Config.Watch() на etcd — hot reload без перезапуска WAFCore
- Config.Rollback() — откат к предыдущей версии правил

9.4 Multi-Tenancy

- TenantID в каждом RequestContext — изоляция правил, лимитов, baseline
- ResourceQuotas: лимиты per-tenant на RPS, количество правил, объём Feature Store
- CrossTenantVisibilityControl: данные одного тенанта недоступны другому

10. Архитектурные решения

Решение	Обоснование	Альтернативы (отклонены)
Go для WAF Core	Latency <10ms: goroutines, zero-copy, статическая компиляция	Java (JVM overhead), Rust (сложнее)
CEL (cel-go) для правил	Типизировано, sandboxed, нет side-effects, JIT не нужен — CEL компилируется при загрузке	Rego/OPA (избыточен для WAF), Lua (небезопасен), custom DSL (дорого)
Python для ML	Богатая экосистема: LightGBM, scikit-learn, PyTorch; изоляция от ядра	Go (ограниченные ML-библиотеки), Java
gRPC для ML-интеграции	Типизированный контракт (protobuf), streaming, Circuit Breaker	REST (нет streaming, overhead JSON), Thrift
ONNX Runtime	Ускорение инференса: 2–5x по сравнению с native Python; vendor-neutral	TensorFlow Serving (heavyweight), TorchServe
LightGBM + Isolation Forest	Быстрый инференс (<5ms), хорошая работа с табличными данными, feature importance	Deep Learning (медленнее, требует GPU), SVM (плохо масштабируется)
Kafka / NATS как MessageBus	Decoupling, event sourcing, replay, SIEM-интеграция; NATS — lightweight альтернатива	Sync HTTP (tight coupling), Redis Pub/Sub (нет persistence)
Redis для State	Latency <1ms, атомарные операции, Sliding Window; Feature Store с EXPIRE	Memcached (нет persistence), PostgreSQL (медленнее)
etcd для конфигурации	Watch API для hot reload, distributed lock, RAFT consensus, HA	Consul (сложнее), file-based (нет hot reload)
Go + Python: два языка	Чёткий контракт через protobuf; каждый язык — в своей зоне ответственности	Только Go (плохие ML-библиотеки), только Python (latency)

11. Требования к качеству

Требование	Целевой показатель	Механизм	Мониторинг
Latency WAFCore p99	< 10 ms	Go goroutines, Redis <1ms, CEL JIT	Prometheus + Jaeger
Latency ML p99	< 25 ms	ONNX Runtime, только серая зона	Jaeger spans
Пропускная способность	100 000+ req/sec/инстанс	Stateless WAFCore, HPA	k6 load tests
ML Precision	> 0.95 SQLi/XSS, > 0.85 BLA	LightGBM + active learning	MLflow
False Positive Rate	< 1%	ML refinement + whitelist	Prometheus alert
Надёжность	99.99% uptime	K8s HA, Circuit Breakers, Failover	AlertManager
ML Coverage	5-15% запросов в ML	Hybrid Decision Engine thresholds	Prometheus counter
Feature Drift	KL-divergence < 0.1	Evidently AI / custom monitor	Ежечасно

12. Риски и технический долг

Риск	Вероятность	Влияние	Митигация
Высокая сложность архитектуры	Высокая	Высокое	Поэтапная реализация; MVP в курсовой, расширение в дипломе
ML частично реализован	Средняя	Среднее	Mock MLService с реалистичными latency; полная реализация в дипломе
Ложные срабатывания (FPR > 1%)	Средняя	Высокое	ML refinement, whitelist, активное тюнирование threshold
Performance regression при добавлении плагинов	Средняя	Высокое	Benchmark в CI; каждый плагин обязан пройти latency-тест
Сложность поддержки Go + Python	Средняя	Среднее	Чёткий protobuf-контракт; независимые деплои; строгое версионирование
Дрейф ML-модели (feature drift)	Средняя	Высокое	KL-divergence мониторинг; auto-retrain pipeline в Airflow
Сложность event-driven дебаггинга	Высокая	Среднее	Distributed tracing (Jaeger), correlation ID во всех событиях
Накопление данных для обучения ML	Средняя	Высокое	Начало с публичных датасетов (CSIC 2010, CICIDS); затем реальный трафик
Интеграции с TI feeds	Низкая	Низкое	Fallback cache; мок-данные для разработки

13. Глоссарий

Термин	Определение
WAF	Web Application Firewall — межсетевой экран для веб-приложений
CEL	Common Expression Language (Google) — типизированный язык выражений, используется для правил WAF
SQLi	SQL Injection — внедрение SQL-кода в запросы к БД
XSS	Cross-Site Scripting — внедрение вредоносных скриптов
SSRF	Server-Side Request Forgery — подделка запросов на стороне сервера
XXE	XML External Entity — атака через XML-парсер
BOLA	Broken Object Level Authorization — OWASP API Top 1; детектируется через ML
BFLA	Broken Function Level Authorization — нарушение авторизации на уровне функций
LightGBM	Gradient Boosting ML-фреймворк (Microsoft); основная модель для классификации атак
Isolation Forest	Unsupervised ML-алгоритм для anomaly detection; быстрый pre-filter
LSTM-Autoencoder	Нейросеть для поведенческого анализа: обнаружение отклонений от baseline
ONNX	Open Neural Network Exchange — формат для кросс-платформенного ML-инференса
Rate Limiting	Ограничение числа запросов; реализовано через Sliding Window в Redis
gRPC	Google RPC — фреймворк для межсервисного взаимодействия (protobuf)
CQRS	Command Query Responsibility Segregation — паттерн разделения команд и запросов
Event Sourcing	Хранение состояния как последовательности событий
mTLS	Mutual TLS — взаимная аутентификация сервисов через сертификаты
RBAC	Role-Based Access Control — управление доступом на основе ролей
IOC	Indicator of Compromise — индикатор компрометации (IP, домен, хэш)
HPA	Horizontal Pod Autoscaler — автомасштабирование подов в Kubernetes
DLQ	Dead Letter Queue — очередь недоставленных сообщений (fallback)
FPR	False Positive Rate — доля легитимных запросов, ошибочно заблокированных
Active Learning	Подход к обучению ML: модель запрашивает разметку для сомнительных примеров